

# Air-Gapped Predictive Copilot for Secure MPLS Operations

An offline AI NOC copilot that predicts SD-WAN-over-MPLS failures before they impact users — and explains why.

---

## Team ICE



# 1. Problem Statement

Modern enterprise and government networks run SD-WAN overlays on top of MPLS underlays to connect branches, hubs, datacentres, and cloud edges. As these networks grow, operational visibility and response speed become critical to service quality and security posture. Conventional NOC tooling remains **reactive** — faults are detected only after users are already affected.

## Two compounding gaps

- **Reactive detection.** Threshold-based alerts fire only after a breach, leaving no time for pre-emptive action.
- **Air-gap constraints.** Regulated and government environments prohibit cloud-connected AI, leaving the most security-sensitive sites without intelligent guidance.

Because the overlay hides the underlay, a problem in the MPLS core — a flapping LSP, a congested hub-spoke link, a drifting controller policy — surfaces as degraded overlay performance only after users feel it. The platform must move NOC operations from reactive to predictive and answer three questions in real time:

| Question                                                   | System output                                       | Component         |
|------------------------------------------------------------|-----------------------------------------------------|-------------------|
| Q1 — What is likely to fail next, and when?                | Fault probability + time-to-impact                  | Predictive Engine |
| Q2 — Why is risk elevated, which signals contributed?      | Grounded explanation + root-cause hypothesis, cited | Copilot           |
| Q3 — What corrective action should be taken before impact? | Prioritised, cited playbook action sequence         | Workflow          |

**Success is forecasting degradation with enough lead time for the NOC to intervene preventively — communicated in operator-ready language, with no dependency on the internet or cloud APIs.**

## 2. Our Solution

**In one sentence: we give a secure, offline network operations centre an assistant that sees trouble coming, explains it in plain English, and never touches the internet.**

Our solution is a single system that an operator runs entirely inside their own secure network. It begins by recreating the network itself — a faithful SD-WAN-over-MPLS simulation with branch, hub, and datacentre sites, the CE, PE, and P devices that carry the traffic, an encrypted IPSec overlay, and a controller that pushes policy — all of which we can deliberately stress with realistic faults. Every device streams telemetry, and a lightweight pipeline collects, normalises, and stores it so the rest of the system works from one consistent view of the network's health.

On top of that data we run a predictive engine. Instead of waiting for a threshold to break, it learns the early signatures of trouble — a queue that is slowly filling, jitter that is creeping upward, a BGP session that is beginning to flap — and turns them into a clear statement: this part of the network is likely to fail, with this probability, in roughly this many minutes. That lead time is the heart of the system, because it is the difference between preventing an outage and merely explaining one after it has happened.

When the engine raises a concern, the workflow layer gathers the related signals, places them on the network's topology graph, and works out the single most likely root cause rather than flooding the operator with separate alarms. That one correlated incident is handed to the copilot — a self-hosted language model that explains what is happening and why, grounded strictly in the team's own runbooks, topology maps, and past incidents. An engineer can simply ask a question in a chatbox and receive an accurate, cited answer, together with the action to take before service is affected.

Crucially, the entire pipeline runs with no internet connection — and we prove it. The system is demonstrated with the network interface disabled and a live capture showing zero outbound traffic. In short, the platform watches the network, sees trouble coming, explains it the way a senior engineer would, and keeps every byte inside the air-gap.

Each part of the platform maps directly onto one of the four objectives of the brief:

| Objective                                | Delivered by |
|------------------------------------------|--------------|
| O1 — Simulated SD-WAN / MPLS environment | Layer 6      |
| O2 — Predictive fault analytics engine   | Layers 4 – 5 |
| O3 — Offline LLM NOC copilot             | Layer 2      |
| O4 — Integrated NOC workflow automation  | Layers 3 & 1 |

### 3. System Architecture

The platform is organised in six layers inside a single air-gapped boundary. Telemetry rises from the simulation through ingestion, prediction, and workflow to a grounded answer for the operator. The entire stack runs on one laptop with zero outbound dependency.

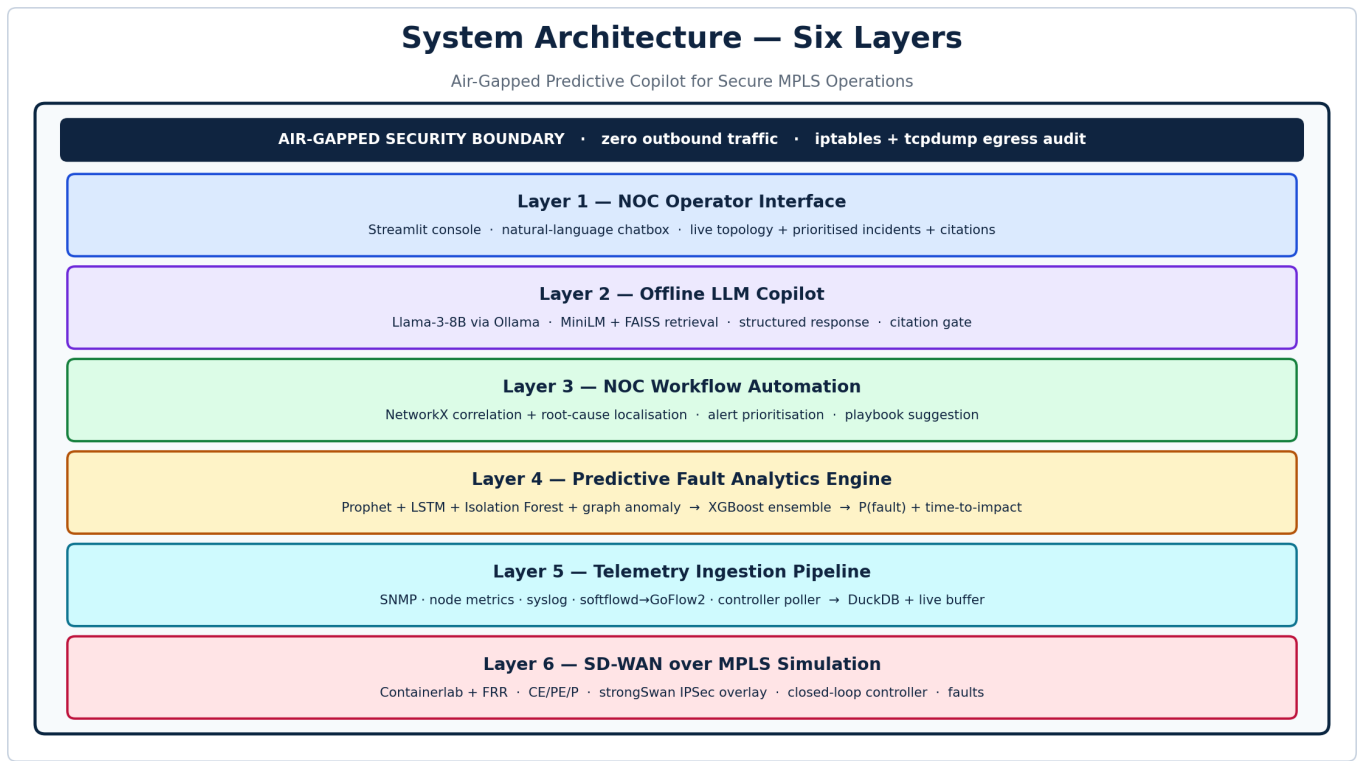


Figure 1 — Six-layer architecture inside the air-gapped boundary.

| Scoring dimension             | Weight | Primarily earned by                             |
|-------------------------------|--------|-------------------------------------------------|
| Technical Merit               | 35%    | Prediction accuracy and lead time               |
| Copilot Effectiveness         | 35%    | Correct, grounded answers with no hallucination |
| Security & Offline Compliance | 20%    | Verifiable zero-egress air-gap                  |
| Documentation Quality         | 10%    | Clarity of design and rationale                 |

## 4. End-to-End Data Flow

The simulation emits telemetry. The pipeline normalises it into DuckDB and an in-memory live buffer. The engine produces a fault probability and time-to-impact. The workflow layer correlates related alerts into a single incident and localises the root cause. The copilot then explains that **correlated incident** — not the raw alert — grounded in retrieved artifacts, and recommends an action. The dashboard presents the prioritised incident to the operator.

What makes this flow powerful is that every stage adds meaning rather than simply passing data along. The counters that enter the pipeline are, on their own, just numbers; by the time they reach the operator they have become a ranked incident with a probable cause, a time-to-impact, and a recommended fix in plain language. And because each step up to the explanation is deterministic — and the explanation itself is constrained to cited evidence — the operator can always trace an answer back through the incident, the prediction, and the exact telemetry that triggered it.

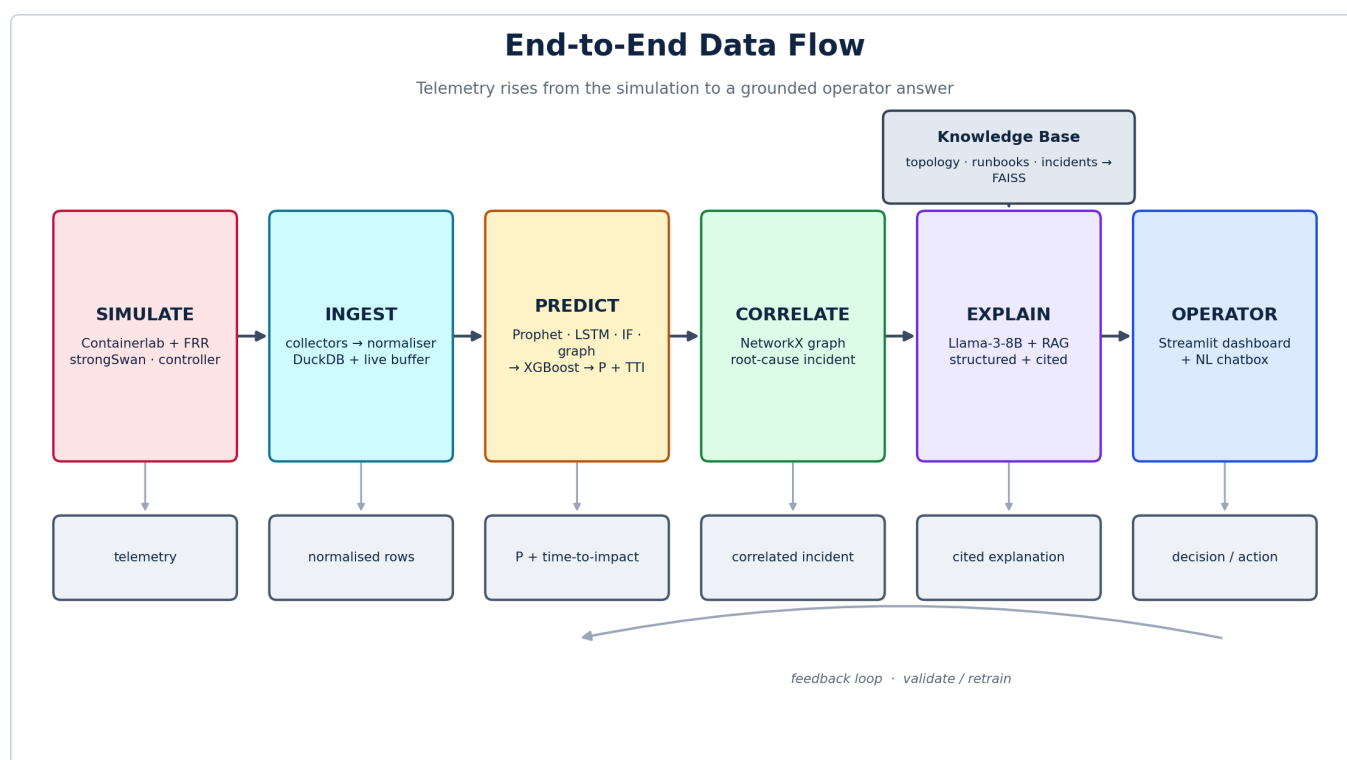


Figure 2 — End-to-end data flow from simulation to a grounded operator answer.

## 5. Simulated SD-WAN over MPLS (Objective 1)

A reproducible multi-site topology built with Containerlab and FRRouting. Customer Edge (CE) routers host the SD-WAN edge at each site; Provider Edge (PE) routers impose MPLS labels and hold VPN routes; the Provider (P) core performs LDP / RSVP-TE label switching. strongSwan builds the IPSec overlay over kernel XFRM, and a closed-loop Python controller applies routing policy via vtysh and QoS via tc — producing real, ground-truth policy drift rather than synthetic labels.

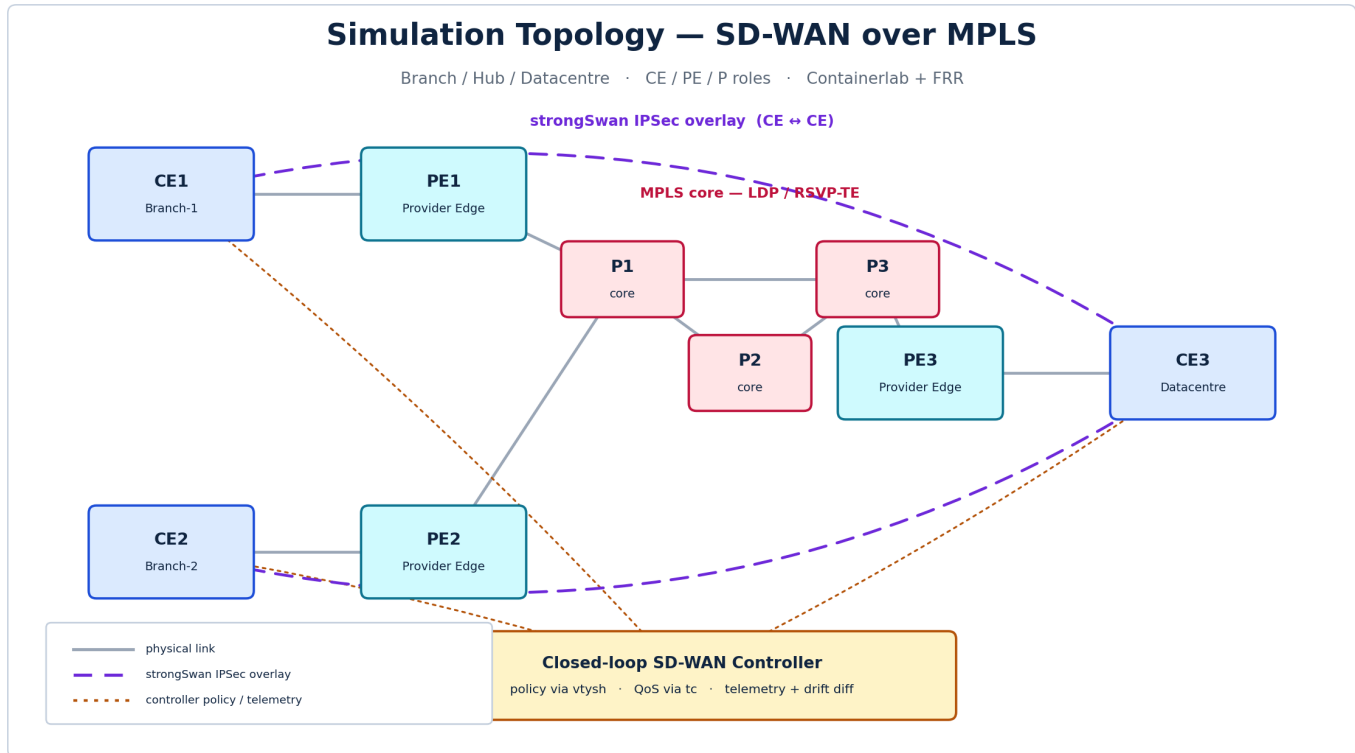


Figure 3 — Branch / Hub / Datacentre topology with CE/PE/P roles, IPSec overlay, and closed-loop controller.

## 6. Telemetry Ingestion Pipeline (Objective 1 → 2)

FRR is a control-plane suite, so the data-plane signals come from purpose-built collectors. All four required sources are normalised and time-aligned into DuckDB for history and feature extraction, while an in-memory recent-window buffer feeds the live dashboard without contending on the database file.

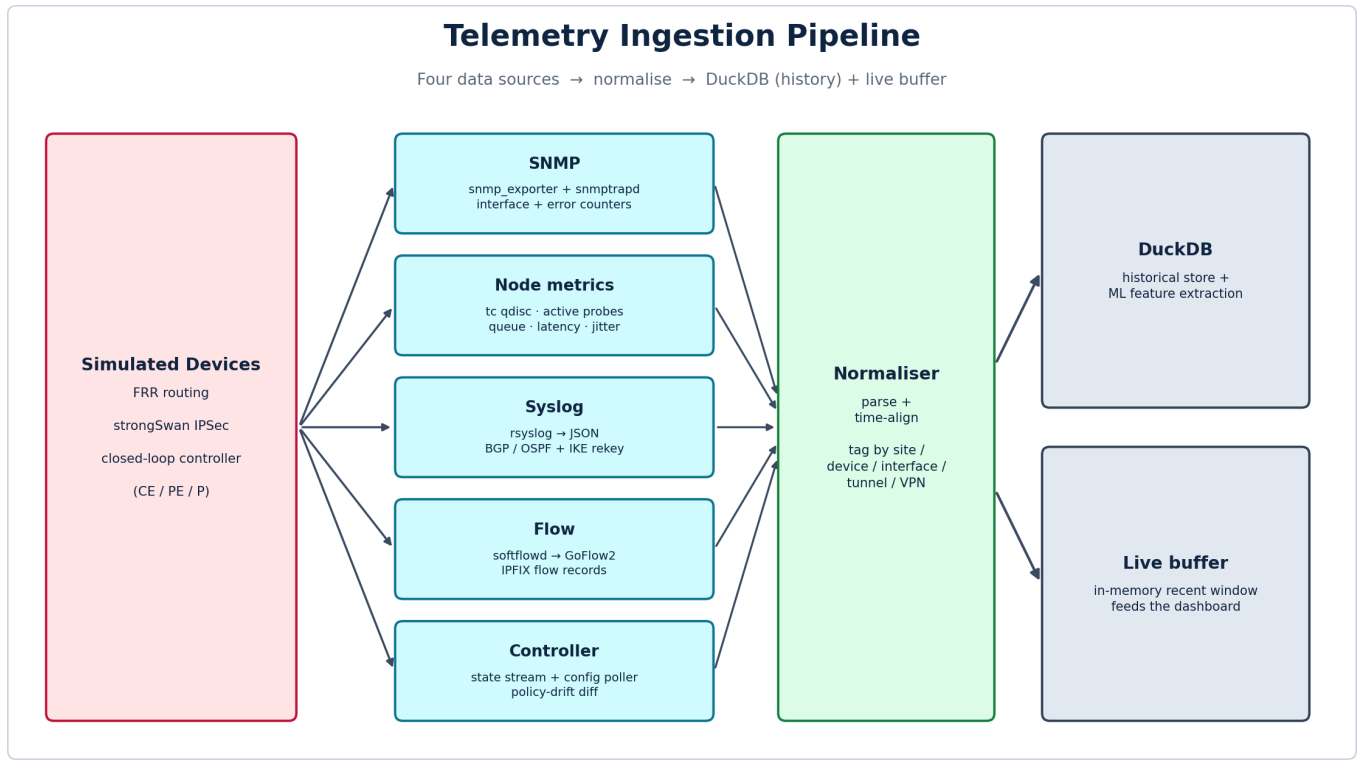


Figure 4 — Four data sources normalised into DuckDB and a live buffer.

| Data source                    | How it is collected                               |
|--------------------------------|---------------------------------------------------|
| Interface utilisation & errors | snmp_exporter + snmptrapd (IF-MIB)                |
| Queue depth, latency, jitter   | node metrics — tc qdisc + active probes           |
| Syslog & routing events        | rsyslog → JSON (BGP/OSPF + strongSwan rekey)      |
| NetFlow / IPFIX flow records   | softflowd → GoFlow2                               |
| Controller policy & drift      | state stream + config poller (intended vs actual) |

# 7. Predictive Fault Analytics Engine (Objective 2)

Precursor features are computed over rolling windows, then four detectors feed a gradient-boosting ensemble that outputs a per-entity fault probability and time-to-impact. The headline metric is **lead time** — how early a fault is forecast before service impact.

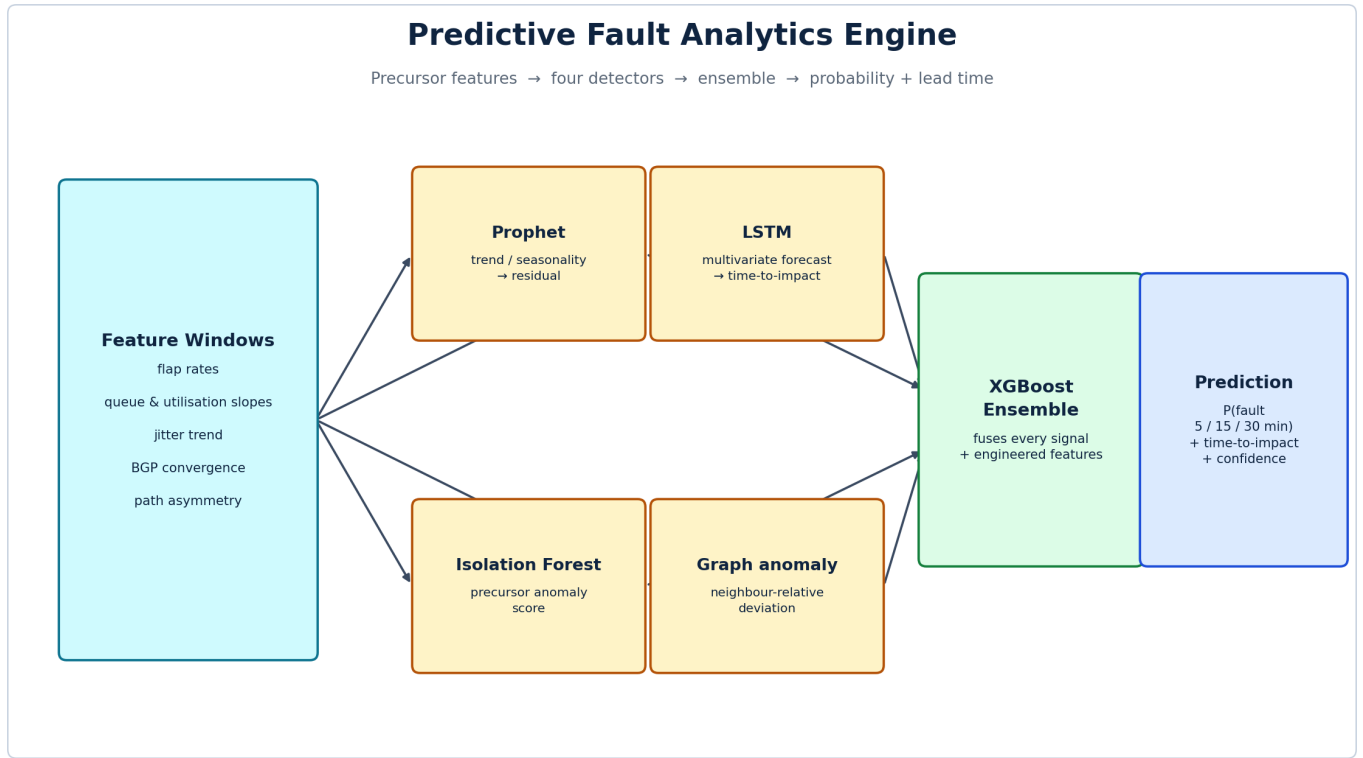


Figure 5 — Four detectors feed the XGBoost ensemble to produce probability and lead time.

| Stage         | Model                                 | Role                                   |
|---------------|---------------------------------------|----------------------------------------|
| Decomposition | Prophet                               | trend / seasonality → residual feature |
| Forecasting   | LSTM (PyTorch)                        | multivariate forecast → time-to-impact |
| Anomaly       | Isolation Forest                      | unsupervised precursor score           |
| Graph anomaly | neighbour-relative + Isolation Forest | topological deviation score            |
| Ensemble      | XGBoost                               | P(fault 5/15/30 min) + confidence      |

**Confidence** is the ensemble's calibrated probability (isotonic regression on a held-out set), reinforced when the detectors agree — never the language model's self-assessment.



## 8. Offline LLM Copilot (Objective 3)

Llama-3-8B runs locally through Ollama. Given the correlated incident, the copilot retrieves relevant internal artifacts from FAISS and produces a structured response. A **citation gate** rejects any answer that cites no artifact, enforcing grounded, no-hallucination output. A NOC engineer asks plain questions in a chatbox and gets an accurate, cited answer without needing to know how to query the network.

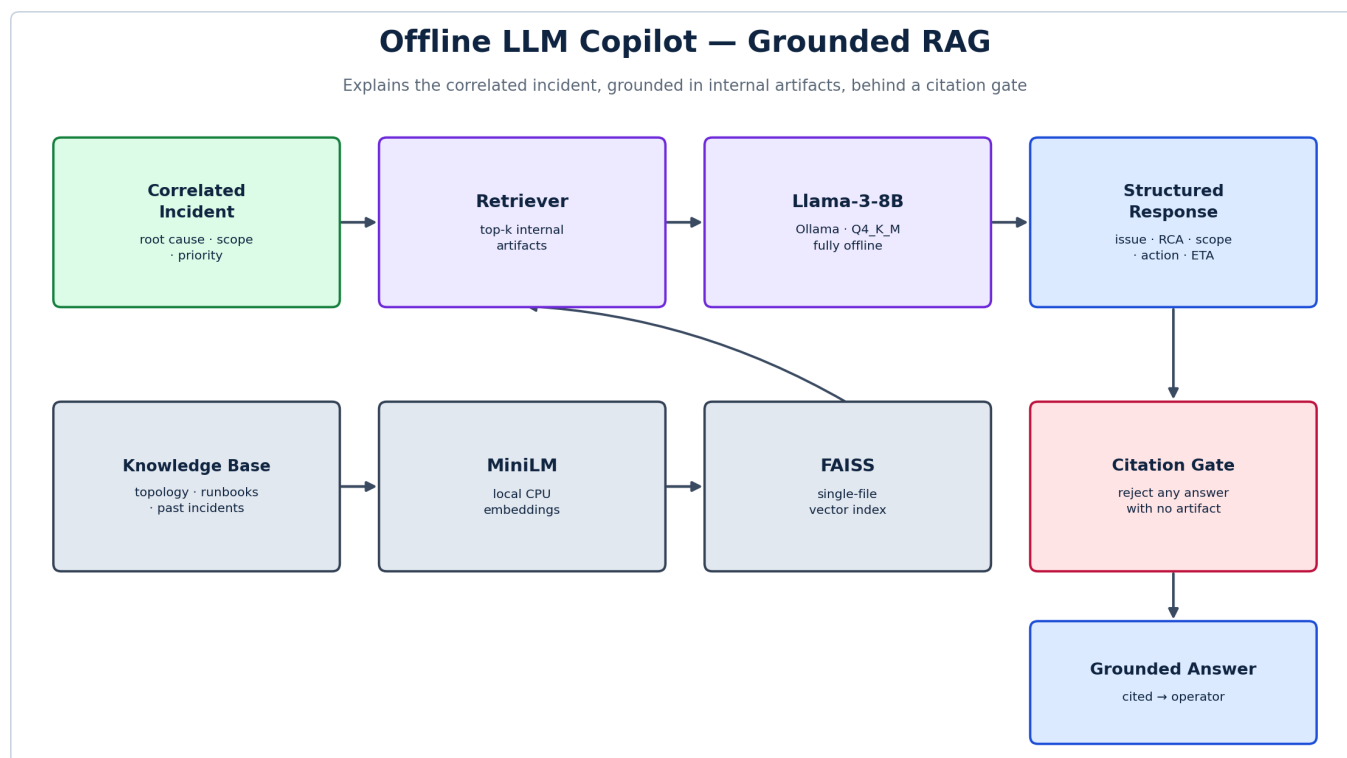


Figure 6 — Grounded RAG: the incident is explained from internal artifacts behind a citation gate.

```
{
  "predicted_issue": "...",
  "confidence": 0.0,
  "root_cause_hypothesis": "...",
  "affected_scope": ["site-A", "tunnel-T3"],
  "estimated_time_to_impact": "14m",
  "recommended_actions": ["...", "..."],
  "cited_artifacts": ["runbook-12", "incident-2024-08"]
}
```

## 9. Integrated NOC Workflow (Objective 4)

Alerts are correlated on the topology graph into a single incident with its root-cause node, ranked by confidence and scope to cut alert fatigue, paired with a suggested playbook, and summarised for the operator. This is where the three operational questions are answered end to end.

This layer is what keeps the operator focused. In a real outage a single underlying fault often sets off dozens of downstream alarms across many devices, and showing them as separate alerts is exactly what causes alert fatigue. By collapsing those related signals onto the topology graph into one incident — with a single root cause and a confidence-ranked priority — the platform turns a noisy flood of symptoms into one clear, actionable item, and hands the operator the matching remediation playbook alongside the copilot's explanation.

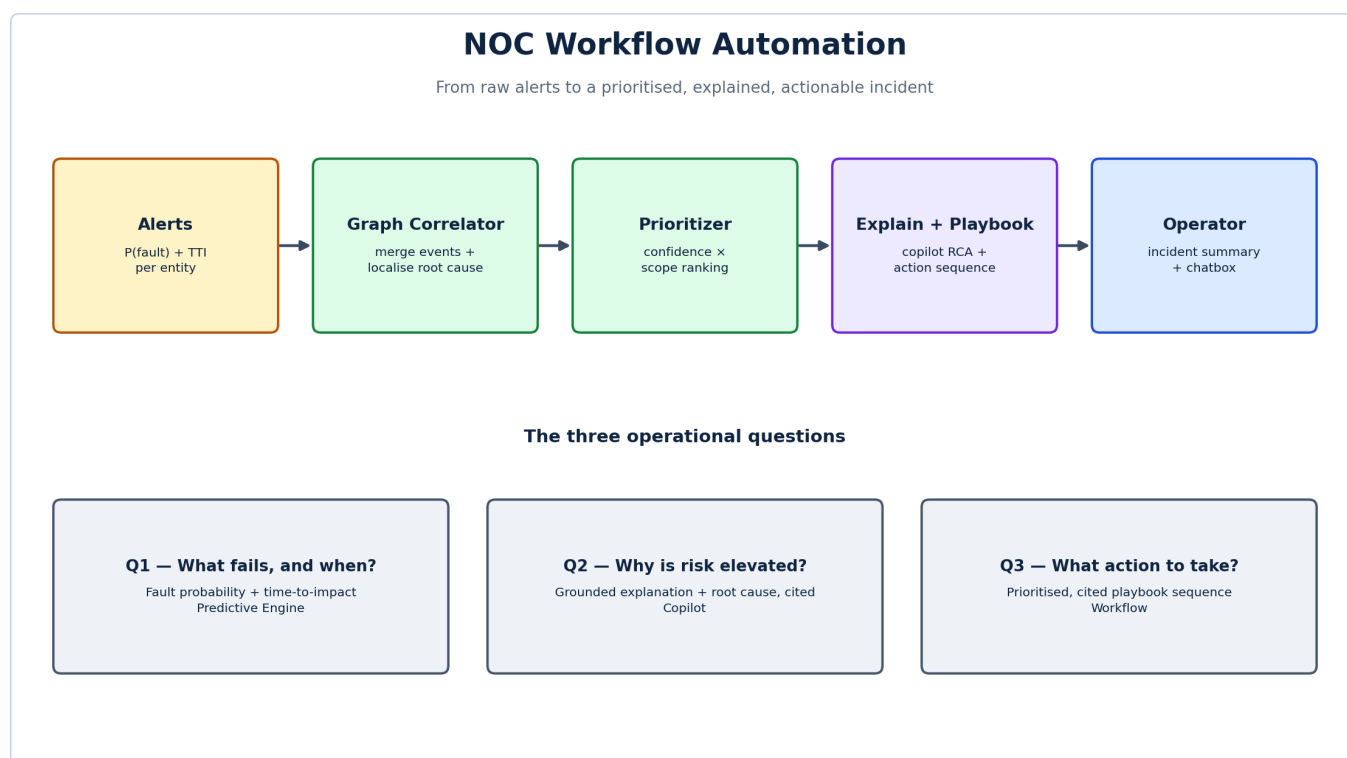


Figure 7 — From raw alerts to a prioritised, explained, actionable incident.

## 10. Technology Stack

Every component runs fully offline and is staged on a golden USB before the finale.

| Layer      | Component           | Technology                                                           |
|------------|---------------------|----------------------------------------------------------------------|
| Simulation | Network simulation  | Containerlab + FRRouting                                             |
| Simulation | SD-WAN overlay      | strongSwan IPSec (kernel XFRM) + closed-loop controller (vtysh + tc) |
| Simulation | Traffic + faults    | iperf3 + tc / netem                                                  |
| Ingestion  | SNMP                | snmp_exporter + snmptrapd                                            |
| Ingestion  | Node metrics        | tc qdisc + active probes                                             |
| Ingestion  | Syslog              | rsyslog → JSON                                                       |
| Ingestion  | Flow                | softflowd → GoFlow2                                                  |
| Ingestion  | Controller          | state stream + config poller                                         |
| Ingestion  | Stores              | DuckDB (history) + in-memory live buffer                             |
| Prediction | Decomposition       | Prophet                                                              |
| Prediction | Forecasting         | LSTM (PyTorch)                                                       |
| Prediction | Anomaly             | Isolation Forest                                                     |
| Prediction | Graph anomaly       | neighbour-relative + Isolation Forest                                |
| Prediction | Ensemble            | XGBoost                                                              |
| Workflow   | Event correlation   | NetworkX graph correlator                                            |
| Copilot    | Embeddings          | all-MiniLM-L6-v2                                                     |
| Copilot    | Vector store        | FAISS                                                                |
| Copilot    | LLM runtime / model | Ollama + Llama-3-8B-Instruct (Q4_K_M GGUF)                           |
| Copilot    | Orchestration       | LangChain (offline)                                                  |
| UI         | Dashboard           | Streamlit                                                            |
| Packaging  | Runtime             | Docker Compose                                                       |

## 11. Validation Scenarios

Each scenario is a reproducible script: fault injected → precursor detected with lead time → copilot explains with citation → playbook suggested.

| # | Scenario                            | Injection method                                             | Key precursor signals                                                      |
|---|-------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------|
| 1 | Progressive hub-spoke congestion    | tc qdisc ramp + staged iperf3 load                           | utilisation & queue slope, drops, jitter, RTT slope                        |
| 2 | BGP route flap → reroute cascade    | cyclic prefix withdraw / clear                               | adjacency changes, advertise/withdraw rate, convergence, path asymmetry    |
| 3 | MPLS underlay + tunnel degradation  | flap core link / RSVP-TE; force strongSwan rekey; netem loss | LSP reroutes, Fast Reroute switchover, IPSec loss, jitter, rekey anomalies |
| 4 | Controller misconfig → policy drift | apply contradictory policy via vtysh / tc                    | policy-vs-device diff, unexpected path changes, QoS drift                  |

For every run we record the prediction lead time, the accuracy of the forecast, the quality of the copilot's explanation, and the relevance of the suggested remediation. Each scenario is executed several times at different intensities, so the models see a balanced range of conditions rather than a single textbook example. Beyond these four headline cases we also inject supporting precursor classes — LDP session flaps, OSPF instability, and link or node failures — and an adversarial case in which a rogue customer-edge router advertises an unauthorised route inside the VPN, so the platform is tested against both everyday degradation and deliberate misbehaviour.

## 12. Security & Air-Gap Compliance

For the deployments this platform targets, the air-gap is not a convenience but a hard rule: a single unexpected outbound connection is a compliance failure, no matter how useful the tool is. We therefore treat zero outbound dependency as something to be demonstrated on demand, never merely claimed.

The air-gap is engineered and proven, not assumed:

- Every dependency — Docker images, Python wheels, model weights, FAISS index, dataset — is staged offline on a golden USB.
- Ollama's update checker and telemetry are disabled in the daemon configuration.
- The demo runs with the network interface down and outbound traffic blocked, allowing loopback only.
- **verify\_airgap.sh** captures egress with tcpdump and ss during a full inference cycle, demonstrating zero outbound traffic.

During the live demonstration this audit runs in front of the judges: the network interface is taken down, outbound traffic is blocked at the firewall, and a packet capture records no egress while the copilot answers a complete query. This turns the hardest requirement of the brief into a visible, repeatable proof rather than a promise on a slide.

# 13. Evaluation Approach

We evaluate on a temporal hold-out (train on earlier time, test on later, no shuffling). Lead time is the gap between actual fault onset and the first time the model crosses the decision threshold. The targets below are the bar we hold the system to during the finale.

We measure lead time as both a mean and a median per scenario, and a prediction that only fires after the fault has already begun earns no lead time at all — so the metric cannot be inflated by late detections. Alongside accuracy we watch the false-positive rate closely, because an assistant that cries wolf is quickly ignored on a real NOC floor.

| Dimension                            | Target                                 |
|--------------------------------------|----------------------------------------|
| Recall at the 15-minute horizon      | $\geq 0.85$ per primary scenario       |
| False-positive rate                  | $\leq 0.10$ per primary scenario       |
| Median lead time                     | $\geq 10$ minutes per primary scenario |
| Hallucination rate                   | 0 (enforced by the citation gate)      |
| Confidence calibration (Brier score) | $\leq 0.10$                            |

The copilot is held to an equally concrete bar. Every answer must cite at least one internal artifact, and a deliberately unanswerable question — such as asking for a metric the system does not collect — must be met with an honest refusal rather than a fabricated reply. Together these targets keep both halves of the system, the forecast and the explanation, measurable and trustworthy.

# 14. 30-Hour Execution Plan

The four-member team works in parallel tracks with sync checkpoints at hours 2, 8, 14, 20, and 24. The critical path — topology → data → models → incident → copilot — is protected first.



## 15. Team ICE

| Member  | Track                            | Responsibilities                                                                                    |
|---------|----------------------------------|-----------------------------------------------------------------------------------------------------|
| Krishna | Network / Simulation / Packaging | Containerlab topology, FRR + strongSwan + controller, ingestion, fault injection, air-gap packaging |
| Ayush   | Predictive Engine                | feature engineering, Prophet / LSTM / Isolation Forest / graph anomaly, XGBoost, evaluation         |
| Anwasha | LLM / RAG Copilot                | runbook corpus, FAISS, Ollama + Llama-3-8B, prompt design, citation gate, air-gap audit             |
| Payal   | Integration / UI / Demo          | Streamlit dashboard, alert-to-copilot wiring, demo flow, pitch deck                                 |

### Why Team ICE

- **Precursor-based lead-time forecasting** — we forecast build-up conditions with a quantified time-to-impact, not threshold breaches.
- **Grounded, no-hallucination copilot** — a deterministic graph correlator localises the root cause and the LLM explains it, every claim cited.
- **Real policy-drift ground truth** — a closed-loop controller generates genuine misconfiguration, so the copilot is validated on real drift, not synthetic labels.
- **Provably air-gapped** — a live zero-egress audit during a full inference cycle.